Reverse Engineering from Code using Agentic AI

1. Project Overview

Software companies often have large and old software projects whose original documentation, design diagrams, and detailed explanations are missing or outdated. Understanding such software requires developers to manually read thousands of lines of code and identify how different files, functions, classes, and modules are connected.

The proposed project, **“Reverse Engineering from Code using Agentic AI,”** aims to develop an AI-based system that automatically analyzes an existing software codebase and converts it into an understandable representation of the software.

The system will accept a source-code project such as a ZIP file or repository and automatically identify the structure, dependencies, function relationships, data flow, control flow, and important modules. Using **Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), program analysis, and LangGraph-based multi-agent orchestration**, the system will generate human-readable explanations and reverse-engineering artifacts.

The main goal is to help developers understand an unfamiliar or legacy software system without manually studying the entire codebase.

---

2. Problem Statement

When developers receive an existing or legacy software project, they may not know:

- What the software actually does.
- Which files belong to which module.
- Which functions call other functions.
- How data moves through the system.
- Which components depend on each other.
- How different modules interact.
- What the purpose of an individual function or class is.
- What the overall architecture of the software looks like.

Traditional reverse-engineering tools can extract structural information, but the information may still be difficult for a developer to understand. On the other hand, an LLM alone may generate incorrect explanations because it does not always have complete knowledge of the relationships within a large codebase.

Therefore, this project proposes a **hybrid and agentic approach** that combines traditional program analysis with LLM-based reasoning and verification.

---

3. Proposed Solution

The proposed system will take an existing source-code repository as input and perform automated software analysis.

The workflow will be:

Source Code → Code Parsing → Program Analysis → Knowledge Representation → RAG → LangGraph Agents → Verification → Reverse-Engineering Output

The system will combine structural information obtained from the code with AI-based semantic understanding.

---

4. How the System Works

Step 1 – Upload Existing Code

The user uploads:

- A ZIP file containing the source code, or
- An existing software repository.

For example:

```
Project
 ├── login.py
 ├── database.py
 ├── payment.py
 ├── user.py
 └── app.py
```

Step 2 – Code Extraction and Identification

The system extracts the project and identifies:

- Programming languages
- Files
- Classes
- Functions
- Methods
- Imports
- Modules

This provides an initial overview of the software.

Step 3 – Structural Code Analysis

The system uses program-analysis techniques to understand the structure of the code.

It can extract:

Abstract Syntax Tree (AST) – shows how the source code is structured.

Call Graph – shows which function calls which other function.

Example:

```
login()
   ↓
validateUser()
   ↓
findUser()
   ↓
database()
```

Dependency Graph – shows relationships between files/modules.

```
app.py
  ↓
login.py
  ↓
database.py
```

Control Flow – identifies possible execution paths.

Data Flow – identifies how important variables and information move through the program.

Step 4 – Create a Software Knowledge Representation

The extracted information is combined into a structured representation of the software.

For example:

```
User
  ↓
Login
  ↓
Authentication
  ↓
Database
```

This allows the AI system to understand not only individual code snippets but also how different parts of the software are related.

Step 5 – RAG-Based Code Retrieval

Large repositories may contain thousands or millions of lines of code. Sending the entire repository to an LLM is inefficient and may exceed its context limits.

Therefore, the system stores relevant code representations as searchable information.

When a user asks:

> "How does the login process work?"

the system retrieves only the relevant files, functions, and relationships.

For example:

```
login.py
authentication.py
user.py
database.py
```

These relevant pieces are then provided to the AI agents.

Step 6 – LangGraph-Based Multi-Agent Analysis

The project will use **LangGraph** to coordinate multiple specialized AI agents.

Instead of using one AI agent for every task, different agents will perform different responsibilities.

Code Understanding Agent

Understands what a function, class, or module does.

Dependency Agent

Identifies relationships between files, classes, modules, and functions.

Summarization Agent

Converts complicated source code into simple natural-language explanations.

Program Analysis Agent

Uses structural information such as control flow, data flow, and program dependencies.

UML/Architecture Agent

Generates diagrams such as class, sequence, dependency, and architecture diagrams.

Program Slicing Agent

Identifies the relevant parts of the program related to a particular variable, function, feature, or operation.

Documentation Agent

Automatically generates documentation for the analyzed codebase.

Verification Agent

Checks whether AI-generated explanations are supported by the actual source code and analysis results.

---

7. Why LangGraph?

LangGraph is used because the proposed system requires a multi-step workflow in which different agents perform different tasks and may need to communicate with each other.

For example:

```
User Question
      ↓
LangGraph Controller
      ↓
Code Retrieval
      ↓
Code Analysis Agent
      ↓
Dependency Agent
      ↓
```

```
Summary Agent
      ↓
Verification Agent
      ↓
Final Answer
```

If the verification agent detects an error, the workflow can return to the relevant agent and regenerate the result.

Therefore, the system is not simply an LLM chatbot; it is an **agentic AI workflow for software analysis**.

---

8. Main Outputs of the System

After analysing the codebase, the system can provide:

Project Overview

A simple explanation of what the software does.

File and Module Explanation

Explains the purpose of important files and modules.

Function and Class Summaries

Explains individual components in understandable language.

Dependency Graph

Shows how different software components are connected.

Call Graph

Shows function-to-function interactions.

Data and Control Flow

Shows how information and execution move through the system.

UML Diagrams

Automatically generates diagrams such as class and sequence diagrams.

Architecture Diagram

Provides a high-level view of the software system.

Program Slices

Shows only the code relevant to a particular function, variable, or feature.

Automatic Documentation

Generates documentation for an existing codebase.

Interactive Code Q&A

Allows the developer to ask questions such as:

> “Where is authentication implemented?”
>
> “Which files are related to payment?”
>
> “What happens when the user clicks Login?”
>
> “Which functions depend on this database method?”

The system answers using the analysed codebase as evidence.

---

9. Example

Suppose the user uploads an old employee-management application.

The system may identify:

```
Frontend
   ↓
Login
   ↓
Authentication
   ↓
Employee Service
   ↓
Database
```

The user can then ask:

“Explain the login process.”

The system analyzes the relevant code and may produce:

```
1. User enters username and password.
2. Login controller receives the request.
3. Authentication service validates the credentials.
4. User information is retrieved from the database.
5. Authentication status is returned.
6. The user is allowed or denied access.
```

The system can also generate the corresponding sequence diagram and identify the source files responsible for each step.

---

10. Research Contribution

Existing research has explored individual tasks such as:

- Code summarization
- Program slicing
- Decompilation
- UML generation
- Code understanding
- Requirements-to-code traceability

However, these tasks are generally addressed separately.

The proposed project aims to combine multiple reverse-engineering capabilities into a **single agentic framework**.

Another important contribution is the use of a **verification-driven architecture**. Since LLMs can generate incorrect or unsupported explanations, the proposed system will use program-analysis evidence and a verification agent to check AI-generated results against the source code.

Thus, the project combines:

Traditional Program Analysis + RAG + LLM + LangGraph Multi-Agent System + Verification

to create a more reliable reverse-engineering platform.

---

11. Target Users

The system can be useful for:

- Software developers
- Maintenance teams
- Developers working with legacy systems
- Software engineering students
- Organizations with poorly documented projects
- Teams taking over projects developed by previous developers

---

12. Expected Outcome

The final system will provide an interactive dashboard where a developer can upload an unfamiliar codebase and obtain a structured and understandable representation of the software.

Instead of manually reading thousands of lines of source code, the developer can use the system to quickly understand:

What the software does → how it is structured → how components interact → how data flows → which code implements a feature → and how the entire system works.

The project therefore aims to reduce the time and effort required for **software maintenance, program comprehension, documentation, and reverse engineering**.